

MVP MILESTONE DESCRIPTION DOCUMENT

Room Revenue Tracker

Django 5 • Python 3.12 • PostgreSQL • DRF

Document Type

Project

Stack

Total Milestones

Version

Date

MVP Milestone & Feature Specification

Room Revenue Tracker — 42 Rooms / 4 Blocks

Python 3.12 / Django 5.x / PostgreSQL / DRF / Celery

7 Milestones across 7 Sprints (~10 weeks)

1.0 — Initial Release

June 2026

1. Purpose & Scope

This document defines the Minimum Viable Product (MVP) milestone plan for the Room Revenue Tracker — a mobile-responsive Django/Python web application that automates revenue management for a 42-room, 4-block residential property (blocks: BBH, NWG, ANX, CRV). It replaces an existing manual, Excel-based workflow with a structured digital system covering tenant management, payment tracking, utility cost splitting, and maintenance triage.

Each milestone is independently deliverable, testable, and deployable. The development team must complete each milestone's acceptance criteria before proceeding to the next. All code must conform to PEP 8, be covered by pytest-django tests, and follow Django best practices including separation of concerns, fat models/thin views, and DRY configuration via environment variables (python-decouple or django-environ).

1.1 Milestone Summary

Milestone	Focus Area	Duration	Sprint	Status
M1 — Project Bootstrap	Django setup, models, auth	1 week	Sprint 1	Pending
M2 — Bed Space & Tenant	Unique ID system, onboarding	1.5 weeks	Sprint 2	Pending
M3 — Revenue Ledger	Payments, proof upload, verification	2 weeks	Sprint 3	Pending
M4 — Utilities Subsystem	Prepaid meters, cost splitting	1.5 weeks	Sprint 4	Pending
M5 — Maintenance Triage	Issue reporting, photo upload	1 week	Sprint 5	Pending
M6 — Dashboards & Reports	Owner/tenant portals, exports	1.5 weeks	Sprint 6	Pending
M7 — QA, Security & Deploy	Testing, hardening, production	1 week	Sprint 7	Pending

1.2 Conventions Used in This Document

- AC — Acceptance Criterion (must pass before milestone sign-off)
- Given/When/Then — BDD-style acceptance criteria format
- Model — Django ORM model class
- View — Django class-based or function-based view
- API endpoint — DRF ViewSet or APIView
- Owner — the property owner / administrator
- Tenant — a person occupying a bed space

2. Technology Stack

All technology choices must remain within the Python/Django ecosystem. No JavaScript frameworks (React, Vue, etc.) are permitted unless pre-approved — HTMX is the approved progressive-enhancement library for frontend interactivity.

Layer	Technology	Purpose
Backend Framework	Django 5.x (Python)	Core MVC framework, ORM, admin panel
REST API	Django REST Framework	Mobile-ready JSON API for all modules
Authentication	django-allauth + JWT	Multi-role auth: owner, tenant, staff
Database	PostgreSQL	Primary RDBMS — relational integrity for rooms/tenants
File Storage	django-storages + S3	Payment proof, maintenance photos, secure upload
Task Queue	Celery + Redis	Async payment verification, reminder notifications
Frontend	Django Templates + HTMX	Mobile-responsive, progressive enhancement
Styling	Tailwind CSS	Utility-first responsive design
Testing	pytest-django	Unit, integration, API tests
CI/CD	GitHub Actions	Automated test + deploy pipeline
Deployment	Gunicorn + Nginx + Docker	Production-grade containerised deployment

2.1 Django App Structure

The project must follow a modular Django app structure. Each functional domain is its own Django app:

```
property_tracker/      # Django project root
  apps/
    core/              # shared models, mixins, middleware
    accounts/          # user auth, roles, profiles
    properties/        # blocks, rooms, bed spaces
    tenants/           # tenant onboarding, contracts
    revenue/           # payments, ledger, verification
    utilities/         # meter readings, cost splitting
    maintenance/       # issue reports, photo uploads
    reports/           # dashboards, exports, analytics
```

- Use Django's AUTH_USER_MODEL with a custom AbstractUser — never extend the default User with a one-to-one profile if the model diverges significantly from the base
- All settings must be split: base.py, development.py, production.py — loaded via DJANGO_SETTINGS_MODULE
- Secret keys, database credentials, and API keys must live in .env — never committed to version control

Milestone 1 — Project Bootstrap & Core Infrastructure

Sprint 1 | Estimated duration: 1 week | Prerequisite: None

Goal

Establish the complete project skeleton, development environment, database schema foundation, multi-role authentication system, and CI/CD pipeline. No business logic ships in this milestone — only the infrastructure every subsequent milestone depends on.

Deliverables

1.1 Project Initialisation

- Django 5.x project created with the app structure defined in Section 2.1
- PostgreSQL configured as the primary database via DATABASE_URL in .env
- python-decouple or django-environ handling all environment configuration
- requirements.txt split into base.txt, development.txt, production.txt
- Pre-commit hooks: black (formatter), flake8 (linter), isort (import sorter)

1.2 Custom User Model & Role System

Implement a custom AbstractUser before the first migration. This is irreversible once data exists — it must be done first.

```
class User(AbstractUser):
    OWNER = 'owner'
    TENANT = 'tenant'
    STAFF = 'staff'
    ROLE_CHOICES = [(OWNER, 'Owner'), (TENANT, 'Tenant'), (STAFF, 'Staff')]
    role = models.CharField(max_length=10, choices=ROLE_CHOICES)
    phone = models.CharField(max_length=20, unique=True)
    block = models.ForeignKey('properties.Block', null=True, blank=True,
                             on_delete=models.SET_NULL)
```

- AUTH_USER_MODEL = 'accounts.User' set before any migrations run
- Three role groups created via data migration: Owner, Tenant, Staff
- django-allauth configured for email + phone login
- JWT tokens via django-rest-framework-simplejwt for API clients

1.3 Core Property Models

```
class Block(models.Model):
    name = models.CharField(max_length=10) # BBH, NWG, ANX, CRV
    code = models.CharField(max_length=5, unique=True)

class Room(models.Model):
    block = models.ForeignKey(Block, on_delete=models.CASCADE)
    number = models.CharField(max_length=10) # e.g. '12'
    capacity = models.PositiveSmallIntegerField(default=2) # 2-4

class BedSpace(models.Model):
```

```
room = models.ForeignKey(Room, on_delete=models.CASCADE)
label = models.CharField(max_length=1) # A, B, C, D
identifier = models.CharField(max_length=20, unique=True,
                             # auto-generated: BBH-12-A
                             editable=False)
is_occupied = models.BooleanField(default=False)
```

The identifier field must be auto-generated in the model's `save()` method using the pattern: `{block.code}-{room.number}-{label}`. This is the system's unique key for every bed space.

1.4 CI/CD Pipeline

- GitHub Actions workflow: run pytest on every pull request to main
- Separate workflow for deployment to staging on merge to main
- Docker + docker-compose.yml for local development parity with production
- Static files via WhiteNoise in production; django-debug-toolbar in development

Acceptance Criteria — Milestone 1

AC #	Given	When	Then
AC-01	The project is freshly cloned	docker-compose up is run	The Django development server starts with no errors and /admin is accessible
AC-02	An owner user is created	The owner logs in via /accounts/login/	They are redirected to a role-appropriate dashboard; the URL is protected from tenant access
AC-03	A Block 'BBH', Room '12', and BedSpace label 'A' exist in the database	The BedSpace is saved	Its identifier field reads exactly 'BBH-12-A'
AC-04	A .env file is missing from the repo	The repository is cloned	No secrets, keys, or credentials are found in any committed file
AC-05	Code is pushed to a pull request	The GitHub Actions CI pipeline runs	All tests pass and coverage is reported

Milestone 2 — Bed Space Management & Tenant Onboarding

Sprint 2 | Estimated duration: 1.5 weeks | Prerequisite: Milestone 1 complete

Goal

Build the complete property management interface: 42 rooms across 4 blocks, bed space unique identification, tenant onboarding with contract assignment, and occupancy tracking. The owner can see every bed space's current status at a glance.

Deliverables

2.1 Block & Room Admin Panel

- Django admin customised with BlockAdmin, RoomAdmin, BedSpaceAdmin
- Bulk creation utility: management command `seed_property` that creates all 42 rooms and their bed spaces from a JSON fixture
- Inline admin for BedSpace within RoomAdmin so the owner can manage spaces in one view
- List display showing: identifier, block, room, label, tenant name, occupancy status

2.2 Tenant Onboarding Flow

```
class TenantProfile(models.Model):
    user = models.OneToOneField(settings.AUTH_USER_MODEL,
                               on_delete=models.CASCADE)
    bed_space = models.ForeignKey(BedSpace, null=True, blank=True,
                                  on_delete=models.SET_NULL)
    move_in_date = models.DateField()
    move_out_date = models.DateField(null=True, blank=True)
    rent_amount = models.DecimalField(max_digits=8, decimal_places=2)
    nrc_number = models.CharField(max_length=20, unique=True)
    emergency_contact = models.CharField(max_length=100)
```

- Owner-only form to create a tenant: name, phone, NRC number, assign bed space
- On tenant creation: User account auto-created, welcome SMS/email sent with login credentials
- BedSpace.is_occupied flips to True on assignment; flips back on vacating
- Validation: a bed space cannot be assigned to more tenants than its Room.capacity allows

2.3 Occupancy Dashboard

- Owner view: grid of all 4 blocks showing each bed space as a coloured card (green=occupied, grey=vacant, amber=notice given)
- Clicking a bed space shows the current tenant's name, move-in date, and outstanding balance
- Mobile-responsive using Tailwind CSS grid — usable on a smartphone
- Tenant view: shows only their own bed space details, block, room, and contract summary

Acceptance Criteria — Milestone 2

AC #	Given	When	Then
AC-01	42 rooms and their bed spaces are seeded	The seed_property command is run	All bed spaces exist with correct identifiers matching the {BLOCK}-{ROOM}-{LABEL} pattern
AC-02	A bed space has capacity 2	The owner attempts to assign a third tenant to it	The system rejects the assignment with a clear validation error
AC-03	A new tenant is created by the owner	The onboarding form is submitted	A User account exists, a TenantProfile is linked, and the bed space is_occupied is True
AC-04	The owner is on the occupancy dashboard	The page is opened on a 375px-wide mobile screen	All 4 blocks and their bed spaces are readable without horizontal scrolling
AC-05	A tenant logs in	They navigate to their profile	They see their identifier, block, room, move-in date, and monthly rent amount only

Milestone 3 — Revenue Ledger & Payment Verification

Sprint 3 | Estimated duration: 2 weeks | Prerequisite: Milestone 2 complete

Goal

Implement the core revenue automation: a digital ledger replacing the Excel spreadsheet, mobile money payment proof upload (Airtel/MTN), and owner verification workflow. This is the highest-priority business module.

Deliverables

3.1 Payment & Ledger Models

```
class PaymentRecord(models.Model):
    PENDING = 'pending'
    VERIFIED = 'verified'
    REJECTED = 'rejected'
    STATUS_CHOICES = [(PENDING, 'Pending'), (VERIFIED, 'Verified'),
                      (REJECTED, 'Rejected')]

    tenant = models.ForeignKey(TenantProfile, on_delete=models.PROTECT)
    bed_space = models.ForeignKey(BedSpace, on_delete=models.PROTECT)
    amount = models.DecimalField(max_digits=8, decimal_places=2)
    payment_method = models.CharField(max_length=10) # airtel / mtn
    transaction_ref = models.CharField(max_length=50)
    proof_image = models.ImageField(upload_to='payments/%Y/%m/')
    status = models.CharField(max_length=10, choices=STATUS_CHOICES,
                             default=PENDING)
    submitted_at = models.DateTimeField(auto_now_add=True)
    verified_at = models.DateTimeField(null=True, blank=True)
    verified_by = models.ForeignKey(settings.AUTH_USER_MODEL,
                                   null=True, on_delete=models.SET_NULL)
    rejection_reason = models.TextField(blank=True)
```

- All proof images stored in S3-compatible bucket via django-storages; never on the server filesystem in production
- Image validation: max 5MB, JPEG/PNG only, checked server-side with Pillow

3.2 Tenant Payment Submission

- Tenant-facing form: amount, payment method (Airtel/MTN), transaction reference number, proof photo upload
- On submission: status = PENDING, owner receives in-app notification and optional email/SMS
- Tenant can view their own payment history with status badges (Pending / Verified / Rejected)
- Rejected payments show the rejection reason and a 'Resubmit' button
- Tenants cannot edit or delete a submitted payment — they can only resubmit a rejected one

3.3 Owner Verification Interface

- Owner sees a queue of all PENDING payments, sortable by block, date, and amount

- Clicking a payment shows the full-size proof image, tenant details, and bed space
- One-click Verify or Reject with optional rejection note
- Verification stamps verified_at and verified_by automatically
- Celery task: 5 days after rent due date, send reminder SMS/email to tenants with PENDING or no payment for the current month

3.4 Revenue Summary

- Owner dashboard widget: total expected revenue this month vs. total verified — shown as a progress bar
- Per-block revenue breakdown table: expected / received / outstanding for BBH, NWG, ANX, CRV
- CSV export of the monthly ledger (all payments with status, tenant, bed space, amount, date)

Acceptance Criteria — Milestone 3

AC #	Given	When	Then
AC-01	A tenant is logged in	They upload a JPG proof of payment with a valid transaction reference	The payment record is created with status=PENDING and the image is stored in S3
AC-02	A payment proof is a 10MB PNG file	The tenant submits it	The form rejects it with an error: 'File too large. Maximum size is 5MB'
AC-03	The owner has a pending payment in the queue	They click Reject and enter a reason	The payment status becomes REJECTED, the tenant sees the reason, and a Resubmit button appears
AC-04	It is 5 days after rent due date	A tenant has no verified payment for the month	A reminder notification is queued via Celery and sent to the tenant's phone/email
AC-05	The owner views the block revenue summary	Block BBH has 10 tenants, 7 verified and 3 pending	The dashboard shows: Expected K10,000 / Verified K7,000 / Outstanding K3,000 (example values)

Milestone 4 — Utilities Subsystem

Sprint 4 | Estimated duration: 1.5 weeks | Prerequisite: Milestone 2 complete

Goal

Build the prepaid meter utility tracking subsystem. Tenants in each block share a utility portal to record meter readings and costs. The system automatically calculates each tenant's share, applies the owner's K70-per-tenant contribution cap, and tracks excess costs that tenants must cover independently.

Business Rules — Critical

- Owner contributes a maximum of K70 per tenant per billing cycle toward utility costs
- Total owner contribution = K70 × number of active tenants in the block
- Excess = total utility cost – owner contribution; this excess is split equally among tenants
- If total cost ≤ (K70 × tenants), owner covers all — tenants pay nothing for utilities that cycle
- Tenants manage this portal independently — the owner does not need to initiate entries
- The K70 cap must be a configurable system setting (SystemConfig model), not hardcoded

Deliverables

4.1 Utility Models

```
class SystemConfig(models.Model):
    owner_utility_cap_per_tenant = models.DecimalField(
        max_digits=6, decimal_places=2, default=70.00)
    class Meta: verbose_name = 'System Configuration'

class MeterReading(models.Model):
    block = models.ForeignKey(Block, on_delete=models.CASCADE)
    reading_date = models.DateField()
    units_used = models.DecimalField(max_digits=8, decimal_places=2)
    cost_per_unit = models.DecimalField(max_digits=6, decimal_places=2)
    total_cost = models.GeneratedField( # or computed in save()
        expression=F('units_used') * F('cost_per_unit'),
        output_field=DecimalField())
    recorded_by = models.ForeignKey(settings.AUTH_USER_MODEL,
                                    on_delete=models.PROTECT)

class UtilityBill(models.Model):
    meter_reading = models.ForeignKey(MeterReading,
    on_delete=models.CASCADE)
    tenant = models.ForeignKey(TenantProfile,
    on_delete=models.CASCADE)
    owner_share = models.DecimalField(max_digits=6, decimal_places=2)
    tenant_excess = models.DecimalField(max_digits=6, decimal_places=2)
    is_paid = models.BooleanField(default=False)
```

4.2 Calculation Service

Implement a pure Python service function (not in a view or model) to keep business logic testable:

```
# apps/utilities/services.py
def calculate_utility_split(meter_reading: MeterReading) ->
list[UtilityBill]:
    config = SystemConfig.objects.get()
    tenants = TenantProfile.objects.filter(
        bed_space__room__block=meter_reading.block,
        move_out_date__isnull=True)
    n = tenants.count()
    owner_contribution = min(config.owner_utility_cap_per_tenant * n,
                             meter_reading.total_cost)
    excess = max(meter_reading.total_cost - owner_contribution, 0)
    per_tenant_excess = excess / n if n > 0 else 0
    # Create UtilityBill for each tenant ...
```

- Service is called from a Django signal (post_save on MeterReading) or explicitly from a view
- UtilityBill records are created atomically within a database transaction (transaction.atomic())

4.3 Tenant Utility Portal

- Tenants in a block share a read-only view of the block's current meter reading and their individual bill
- Any tenant in the block can submit a new meter reading (recorded_by tracks who submitted it)
- Tenants see: total cost, owner share, their excess amount, and whether their bill is paid
- Tenants can mark their own excess as paid (triggers a simplified payment record — no proof upload required for utilities)

Acceptance Criteria — Milestone 4

AC #	Given	When	Then
AC-01	Block BBH has 10 active tenants, total utility cost is K900	A meter reading of K900 is submitted	Owner contribution = K700 (K70 × 10); excess = K200; each tenant owes K20
AC-02	Block NWG has 5 tenants, total utility cost is K200	A meter reading of K200 is submitted	Owner contribution = K200 (cap is K350 but cost is lower); tenant excess = K0; tenants pay nothing
AC-03	The owner changes the cap to K80 in SystemConfig	A new meter reading is submitted	All future bills use K80 as the cap; existing bills are unchanged
AC-04	A MeterReading is saved	The post_save signal fires	UtilityBill records are created for every active tenant in the block within a single database transaction
AC-05	The calculate_utility_split service is tested in isolation	pytest runs	The function returns correct owner_share and tenant_excess values for given inputs without touching the database

Milestone 5 — Maintenance Triage Module

Sprint 5 | Estimated duration: 1 week | Prerequisite: Milestone 2 complete

Goal

Allow tenants to report maintenance issues directly from their portal with photo evidence. Issues are automatically tagged to their bed space and routed to the owner for triage and resolution tracking.

Deliverables

5.1 Maintenance Models

```
class MaintenanceIssue(models.Model):
    OPEN = 'open'
    IN_PROGRESS = 'in_progress'
    RESOLVED = 'resolved'
    CLOSED = 'closed'
    STATUS_CHOICES = [...]

    PLUMBING = 'plumbing'
    ELECTRICAL = 'electrical'
    STRUCTURAL = 'structural'
    APPLIANCE = 'appliance'
    OTHER = 'other'
    CATEGORY_CHOICES = [...]

    bed_space = models.ForeignKey(BedSpace, on_delete=models.PROTECT)
    reported_by = models.ForeignKey(settings.AUTH_USER_MODEL,
                                    on_delete=models.PROTECT)
    category = models.CharField(max_length=15,
                                choices=CATEGORY_CHOICES)
    title = models.CharField(max_length=100)
    description = models.TextField()
    status = models.CharField(max_length=15, choices=STATUS_CHOICES,
                              default=OPEN)
    created_at = models.DateTimeField(auto_now_add=True)
    resolved_at = models.DateTimeField(null=True, blank=True)
    resolution_note = models.TextField(blank=True)

class IssuePhoto(models.Model):
    issue = models.ForeignKey(MaintenanceIssue, on_delete=models.CASCADE,
                              related_name='photos')
    image = models.ImageField(upload_to='maintenance/%Y/%m/')
    uploaded_at = models.DateTimeField(auto_now_add=True)
```

5.2 Tenant Reporting Interface

- Tenant form: category dropdown, title, description, and up to 3 photo uploads
- Bed space is auto-populated from the logged-in tenant's TenantProfile — no manual selection required (this is the 'automatic bed-space tagging')
- Photos: max 5MB each, JPEG/PNG only, validated server-side
- On submission: issue created with status=OPEN, owner notified via in-app notification
- Tenant can track the status of their own submitted issues

- Tenants cannot edit an issue after submission — they can add a follow-up comment

5.3 Owner Triage Interface

- Owner sees all open issues in a table: bed space ID, category, title, date, status
- Filterable by block, category, and status — sortable by date and priority
- Clicking an issue shows full description, all photos (lightbox display), and a status update form
- Owner can update status (Open → In Progress → Resolved → Closed) with a resolution note
- Resolved status auto-stamps resolved_at; tenant receives notification
- Summary widget on owner dashboard: open issues count by block

Acceptance Criteria — Milestone 5

AC #	Given	When	Then
AC-01	A tenant with bed space BBH-12-A is logged in	They submit a maintenance report	The created MaintenanceIssue.bed_space identifier is 'BBH-12-A' — set automatically, not by the tenant
AC-02	A tenant submits a report with 4 photos	The form is submitted	The system rejects it with 'Maximum 3 photos per report'
AC-03	An issue is in OPEN status	The owner updates it to RESOLVED with a note	resolved_at is stamped, the tenant sees the resolution note, and status shows RESOLVED
AC-04	The owner views the maintenance triage page	They filter by Block NWG and Category Plumbing	Only NWG plumbing issues appear in the list
AC-05	A photo upload is a 12MB JPEG	The tenant submits the form	Server-side validation rejects it with a file size error before any data is saved

Milestone 6 — Dashboards, Portals & Reporting

Sprint 6 | Estimated duration: 1.5 weeks | Prerequisite: Milestones 3, 4, 5 complete

Goal

Consolidate all modules into polished, role-specific dashboards. The owner gets a full-property overview and data export tools. Tenants get a clean personal portal showing everything relevant to them in one place.

Deliverables

6.1 Owner Dashboard

- KPI summary row: Total rooms / Occupied / Vacant / Revenue this month (verified) / Outstanding / Open maintenance issues
- Block-by-block occupancy grid (from Milestone 2) — live, colour-coded
- Revenue progress bars per block: expected vs. verified this month
- Recent activity feed: last 10 payments submitted, last 5 issues reported
- Pending verification queue: count badge linking to unverified payments
- Monthly revenue chart: 12-month bar chart of total verified revenue (using Django's ORM aggregation — no external charting library unless it is Chart.js via CDN)

6.2 Tenant Portal

- Personal header: tenant name, bed space identifier, block, move-in date
- Balance summary: rent due this month, last payment status, any outstanding utility excess
- Quick actions: Submit Payment, Report Issue, View Utility Bill
- Payment history: last 6 months with status badges and amounts
- Active maintenance issues: status and last update for issues they reported
- Utility section: current block meter reading, their share, paid/unpaid status

6.3 Reports & Data Export

- Monthly Ledger Export: CSV of all payments (bed space, tenant, amount, method, status, date) — date-range filterable
- Occupancy Report: CSV of all bed spaces with current tenant and vacancy status
- Utility Report: CSV of all meter readings and per-tenant bills for a selected period
- Maintenance Report: CSV of all issues with status, category, resolution time
- All exports generated as streaming responses (StreamingHttpResponse) to handle large datasets without memory issues

Acceptance Criteria — Milestone 6

AC #	Given	When	Then
AC-01	The owner has verified 15 of 20 expected payments this month	They view the dashboard	The revenue section shows: Expected K20,000 / Verified K15,000 / Outstanding K5,000 with a 75% progress bar

AC-02	A tenant logs in	They view their portal	They see their bed space ID, current rent status, last payment result, utility bill, and any open maintenance issues in a single view
AC-03	The owner requests a monthly ledger export for May 2026	They click 'Export CSV'	A CSV file downloads with all payment records for May 2026 — one row per payment
AC-04	The tenant portal is opened on a 375px mobile screen	The layout is inspected	All sections stack vertically, no horizontal overflow, and touch targets are at least 44px
AC-05	The owner dashboard loads	The ORM query count is measured with django-debug-toolbar	The dashboard page makes no more than 15 database queries (N+1 queries are eliminated via select_related/prefetch_related)

Milestone 7 — QA, Security Hardening & Production Deployment

Sprint 7 | Estimated duration: 1 week | Prerequisite: All milestones complete

Goal

Conduct full test coverage review, security audit, performance optimisation, and deploy to a production environment. This milestone produces a live, hardened application ready for real tenant and owner use.

Deliverables

7.1 Test Coverage

- Minimum 80% test coverage across all apps (measured by pytest-cov)
- All service functions (calculate_utility_split, etc.) must have 100% unit test coverage
- Integration tests for all critical user journeys: tenant submits payment → owner verifies → ledger updates
- API tests via DRF's APIClient for all REST endpoints
- Factory Boy used for all test fixtures — no hardcoded test data

7.2 Security Hardening

- django-axes: account lockout after 5 failed login attempts
- HTTPS enforced: SECURE_SSL_REDIRECT=True, HSTS headers set
- CSRF protection verified on all POST forms
- Content Security Policy headers via django-csp
- File upload security: all uploads scanned for MIME type mismatch; images re-saved through Pillow to strip embedded metadata/scripts
- Rate limiting on payment submission and login endpoints (django-ratelimit)
- SECRET_KEY rotated for production; DEBUG=False enforced
- Run django-check-security and resolve all warnings before go-live

7.3 Performance

- All list views paginated (20 items per page default, configurable)
- Database indexes added on: BedSpace.identifier, PaymentRecord.status + submitted_at, MaintenanceIssue.status + bed_space
- select_related / prefetch_related applied to eliminate N+1 queries on all dashboard views
- Static files served via WhiteNoise with compression enabled
- Celery beat scheduler configured for automated reminders (payment due notices, etc.)

7.4 Production Deployment

- Docker image built and pushed to a container registry
- Gunicorn (4 workers minimum) behind Nginx as reverse proxy
- PostgreSQL managed database provisioned with automated daily backups
- Environment variables loaded from a secure secrets manager (not .env files in production)

- Health check endpoint: GET /health/ returns HTTP 200 with DB connectivity status
- Error tracking via Sentry (free tier acceptable for MVP)
- Deployment runbook documented: how to deploy, rollback, and run migrations

Acceptance Criteria — Milestone 7

AC #	Given	When	Then
AC-01	All tests are run	pytest-cov generates a coverage report	Overall coverage is $\geq 80\%$; <code>calculate_utility_split</code> coverage is 100%
AC-02	The production site is accessed over HTTP	The browser follows the request	HTTP redirects to HTTPS automatically; HSTS header is present in the response
AC-03	A bot attempts to brute-force the login page	More than 5 failed attempts occur from one IP	The account is locked and further attempts return HTTP 403 for 15 minutes
AC-04	The application is in production	Gunicorn crashes with an exception	Sentry captures the error and an alert is triggered; the health check endpoint reflects the failure
AC-05	A tenant attempts to view another tenant's payment proof URL directly	They paste the S3 URL into their browser	The URL is either expired (signed URL) or returns HTTP 403 — the file is not publicly accessible

Appendix A — Definition of Done

A milestone is considered DONE when ALL of the following conditions are met:

1. All Acceptance Criteria for the milestone pass without exception
2. All code has been peer-reviewed and merged to the main branch via pull request
3. Test coverage for new code meets or exceeds the project minimum (80%)
4. No critical or high-severity security issues remain open
5. The feature works correctly on Chrome (mobile), Safari (mobile), and Firefox (desktop)
6. All new models have been migrated and migrations are committed to version control
7. Django admin is configured for all new models (at minimum `list_display` and `search_fields`)
8. The staging environment has been updated and manually verified

Appendix B — Coding Standards

- Follow PEP 8 — enforced by flake8 and black in pre-commit hooks
- Use type hints on all service functions and complex model methods
- No raw SQL unless performance-justified and documented — use the Django ORM
- All user-facing strings must be wrapped in `gettext_lazy()` for future i18n readiness
- Use Django's messages framework for all user feedback (success, error, warning)
- Class-based views preferred over function-based views for complex logic
- All forms must inherit from `django.forms.ModelForm` where a model is involved
- Never use `print()` for debugging — use Python's logging module
- All Celery tasks must be idempotent — running twice must not create duplicate records

Appendix C — Environment Variables Reference

```
# Core
SECRET_KEY=<generated-secret>
DEBUG=False
ALLOWED_HOSTS=yourdomain.com,www.yourdomain.com

# Database
DATABASE_URL=postgres://user:pass@host:5432/dbname

# File Storage
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_STORAGE_BUCKET_NAME=
AWS_S3_REGION_NAME=

# Redis / Celery
REDIS_URL=redis://localhost:6379/0
CELERY_BROKER_URL=redis://localhost:6379/1
```

```
# Email / SMS
EMAIL_HOST=smtp.yourmailprovider.com
EMAIL_HOST_USER=
EMAIL_HOST_PASSWORD=
SMS_API_KEY=

# Sentry
SENTRY_DSN=
```

Appendix D — Recommended Third-Party Packages

Package	Version	Purpose
django-rest-framework	3.15+	REST API layer
django-rest-framework-simplejwt	5.x	JWT authentication for API clients
django-allauth	0.63+	Multi-provider authentication
django-storages[s3]	1.14+	S3-compatible file storage backend
celery[redis]	5.x	Async task queue
django-axes	6.x	Brute-force login protection
django-csp	3.x	Content Security Policy headers
django-ratelimit	4.x	Rate limiting on views
django-environ	0.11+	Environment variable management
Pillow	10.x	Image processing and validation
pytest-django	4.x	Test framework
pytest-cov	4.x	Test coverage reporting
factory-boy	3.x	Test fixture generation
sentry-sdk	2.x	Error tracking in production
whitenoise	6.x	Static file serving
gunicorn	22.x	WSGI production server